

Introduction to Algorithm

기초 알고리즘

알고리즘 분석을 위한 수학 (1)

완전 탐색



가능한 모든 경우를 탐색하는 기법

브루트 포스(Brute-Force)로도 불림



무차별 대입(Brute-Force)



예시

- 크기가 N 인 배열이 있고, 배열 안의 요소들은 모두 양의 정수이며, 오름차순으로 정렬되어 있다. 이때, 배열 안에서 중복되지 않게 두 개를 골라 곱하였을 때의 최대값을 구하여라.
- 두 수를 선택하는 경우의 수는 $\frac{N \times (N - 1)}{2}$
 - 시간 복잡도: $O(N^2)$
 - 모든 경우의 수 탐색 → 완전 탐색
- 배열의 가장 뒤에 있는 요소와 2번째로 뒤에 있는 요소를 곱함
 - 시간 복잡도: $O(1)$
 - 모든 경우의 수를 탐색하지 않음 → 그리디



완전 탐색의 중요성

문제가 주어지면,
무.조.건.

완전 탐색법으로 먼저 시도해야 한다.



문제 해결의 흐름

A라는 방법으로 시도해봤더니 안되더라.

왜냐하면 a라는 이유 때문에.

따라서 a를 해결하는 B라는 방법으로 해봤더니 안되더라.

왜냐하면 b라는 이유 때문에.

...

따라서 f를 해결하는 G라는 방법으로 해봤더니 되더라!!!



문제 해결의 흐름

완전 탐색

A라는 방법으로 시도해봤더니 안되더라.

왜냐하면 a라는 이유 때문에.

따라서 a를 해결하는 B라는 방법으로 해봤더니 안되더라.

왜냐하면 b라는 이유 때문에.

...

따라서 f를 해결하는 G라는 방법으로 해봤더니 되더라!!!



[예제 1] 연속 부분 최대 합

입력 예시

1 2 -4 5 3 -2 9 10

출력 예시

15

연속된 부분을 선택하였을 때, 그 최대 합을 출력

단, $1 \leq n \leq 100$



[예제 1] 연속 부분 최대 합

연속된 부분을 선택하였을 때, 그 최대 합을 출력

1	2	-4	5	3	-2	9	10
---	---	----	---	---	----	---	----

[예제 1] 연속 부분 최대 합

연속된 부분을 선택하였을 때, 그 최대 합을 출력

1	2	-4	5	3	-2	9	10
---	---	----	---	---	----	---	----

완전 탐색 : 가능한 모든 연속 구간을 모두 고려하자

가능한 모든 연속 구간이 몇개나 있나 ?

[예제 1] 연속 부분 최대 합

연속된 부분을 선택하였을 때, 그 최대 합을 출력

1	2	-4	5	3	-2	9	10
---	---	----	---	---	----	---	----

↑
start

↑
end

연속 부분은 (start, end) 로 정의됨

총 28개의 경우가 있음

= 총 **$O(n^2)$** 개의 경우가 있음

[예제 1] 연속 부분 최대 합

연속된 부분을 선택하였을 때, 그 최대 합을 출력

1	2	-4	5	3	-2	9	10
---	---	----	---	---	----	---	----

↑
start

↑
end

한 개의 구간에 대하여 합을 구하는 데 걸리는 시간은 ?

전체 시간 : (가능한 경우의 수) x (하나의 경우에 걸리는 시간)

[예제 1] 연속 부분 최대 합

연속된 부분을 선택하였을 때, 그 최대 합을 출력

1	2	-4	5	3	-2	9	10
---	---	----	---	---	----	---	----

↑
start

↑
end

한 개의 구간에 대하여 합을 구하는 데 걸리는 시간은 ? **$O(n)$**

전체 시간 : (가능한 경우의 수) x (하나의 경우에 걸리는 시간)

$O(n^3)$

O(n²)

O(n)



[예제 2] 멱집합 구하기

입력 예시

3

n개의 원소를 가지는 집합의 멱집합 구하기
단, $1 \leq n \leq 10$

출력 예시

```
1
1 2
1 2 3
1 3
2
2 3
3
```



[예제 3] 균형 맞추기

입력 예시

1 -3 4 5 -2

출력 예시

1

n개의 숫자를 두 개의 그룹으로 나누어, 그 합을 가장 가깝게 하라

단, $1 \leq n \leq 10$



[심화 예제] 뒤집기

입력 예시

```
4 3
0 1 0
1 0 1
1 0 1
0 1 0
```

출력 예시

```
2
```

한 칸을 뒤집으면 상,하,좌,우 색깔이 바뀜

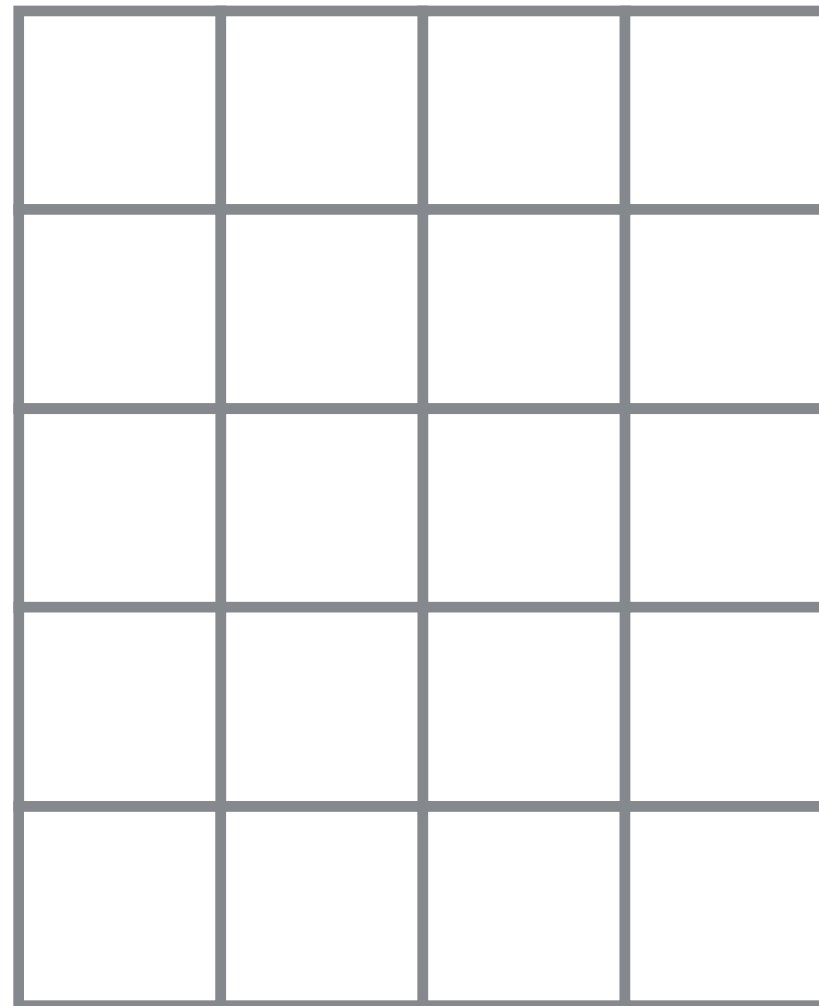
모든 칸의 색이 서로 같아지도록 만들기 위해 최소로 뒤집는 횟수는 ?

단, $1 \leq n \leq 1000$, $1 \leq m \leq 1000$



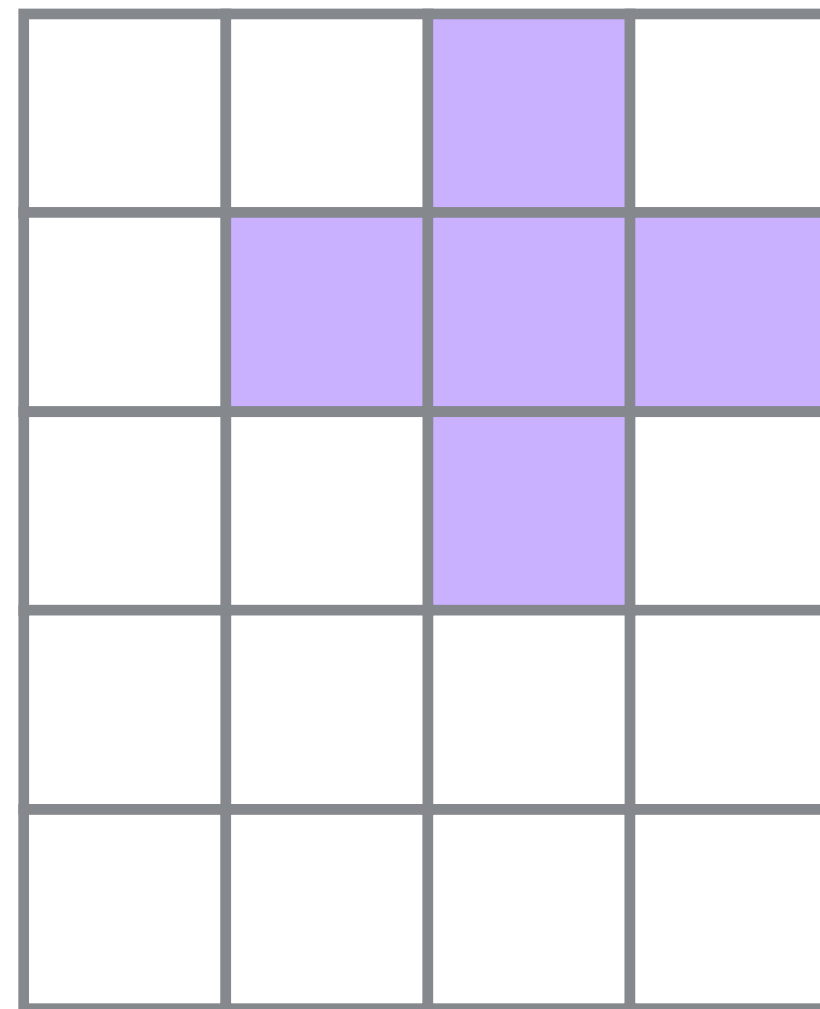
[심화 예제] 뒤집기

관찰 1 : 한 칸을 두 번 뒤집을 필요가 없다



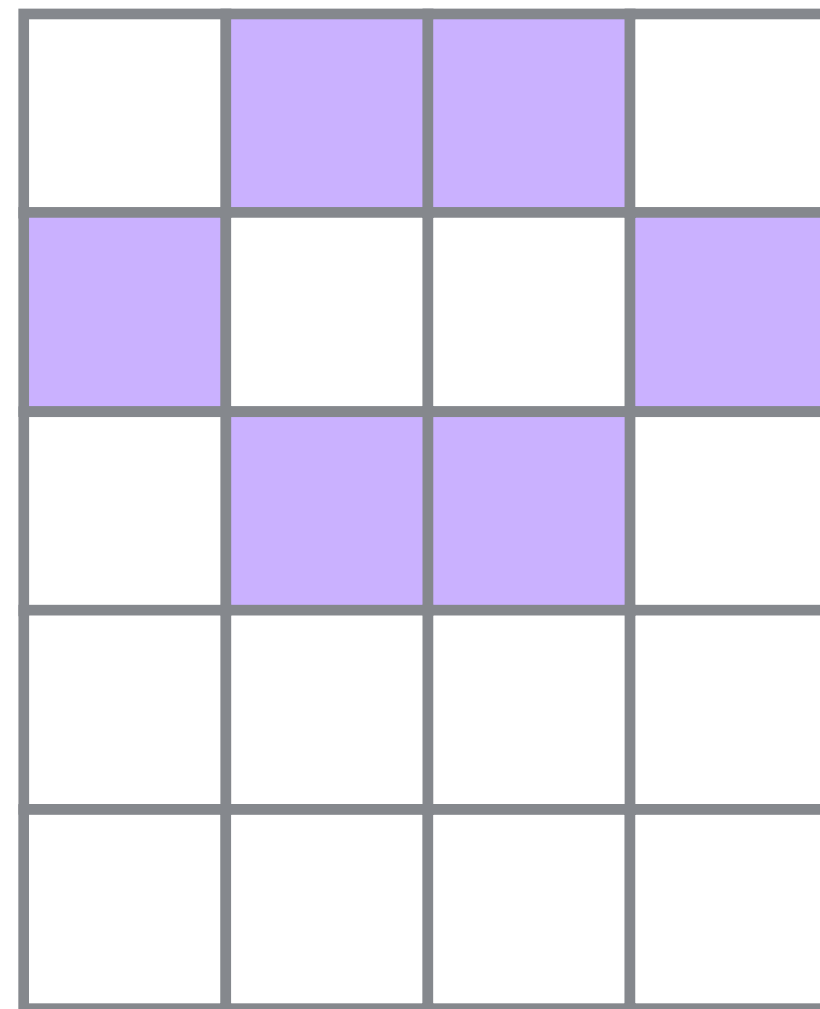
[심화 예제] 뒤집기

관찰 1 : 한 칸을 두 번 뒤집을 필요가 없다



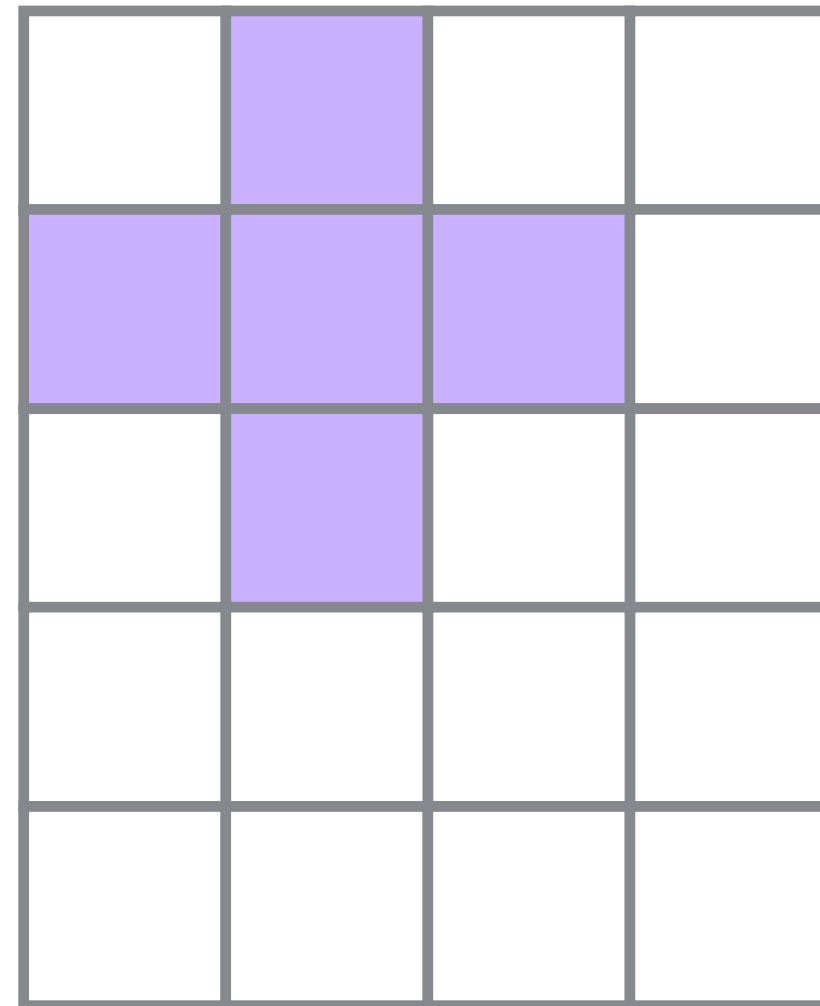
[심화 예제] 뒤집기

관찰 1 : 한 칸을 두 번 뒤집을 필요가 없다



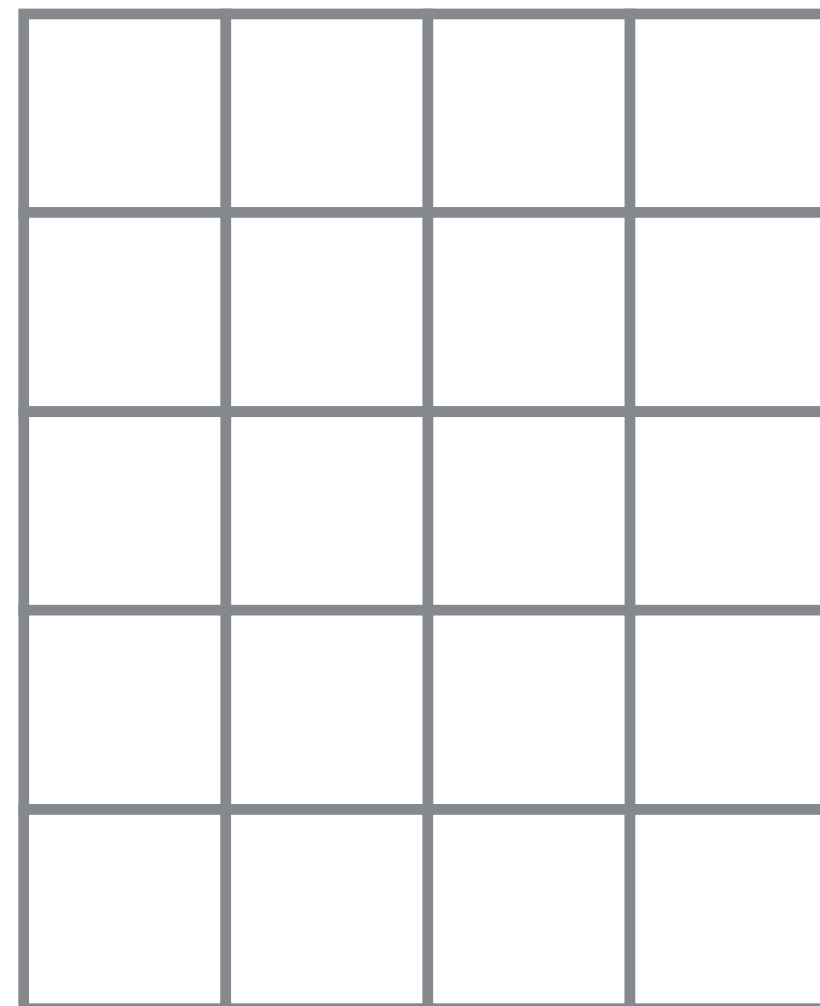
[심화 예제] 뒤집기

관찰 1 : 한 칸을 두 번 뒤집을 필요가 없다



[심화 예제] 뒤집기

관찰 1 : 한 칸을 두 번 뒤집을 필요가 없다



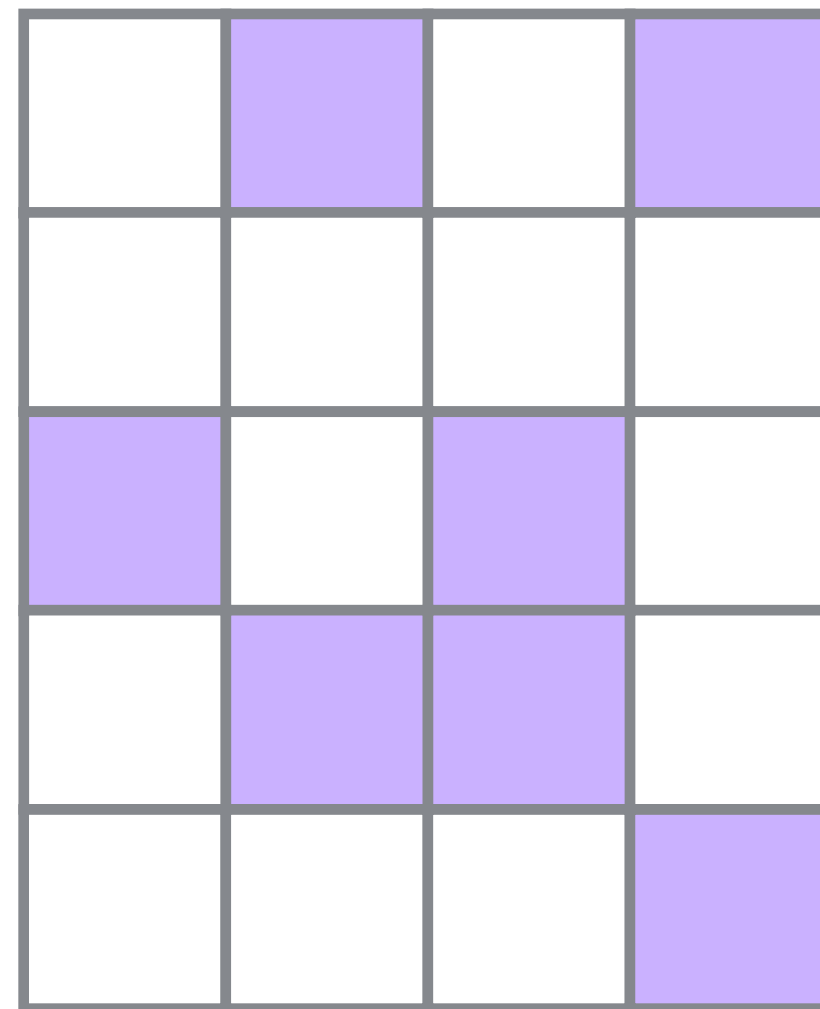
각 칸에 대하여
2가지 경우가 있음

: 누르거나, 누르지 않거나

$O(2^{NM})$ 가지의 경우가 존재 : 너무 많다

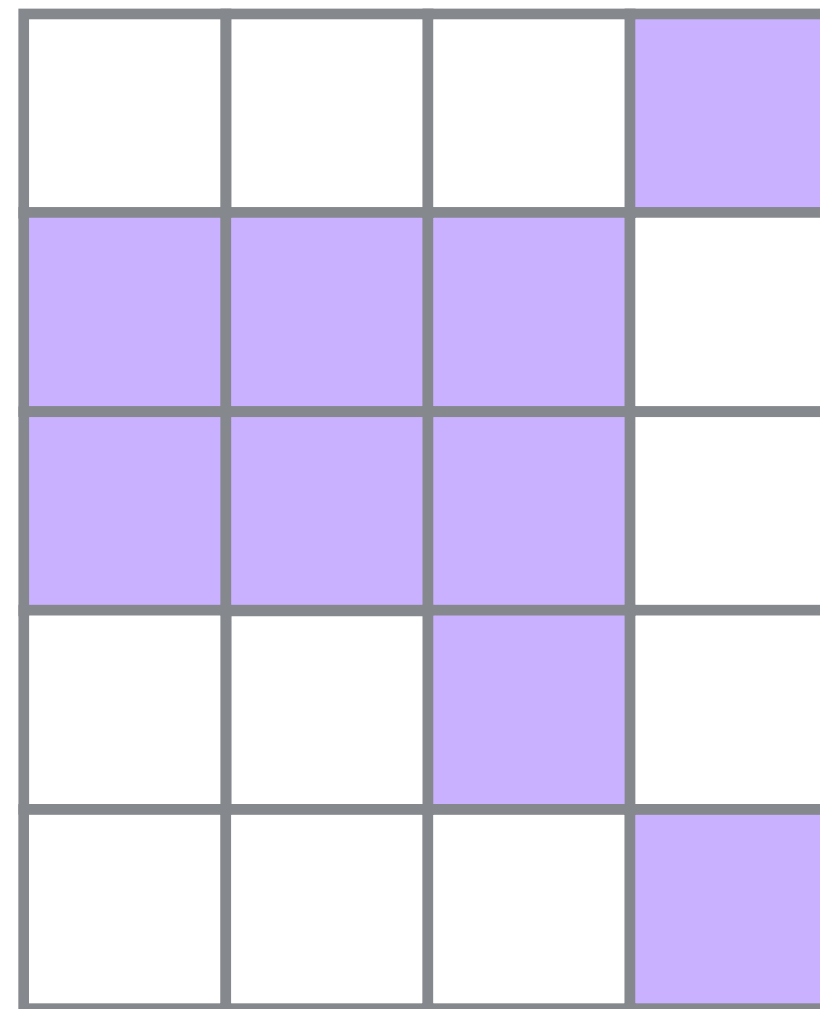
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



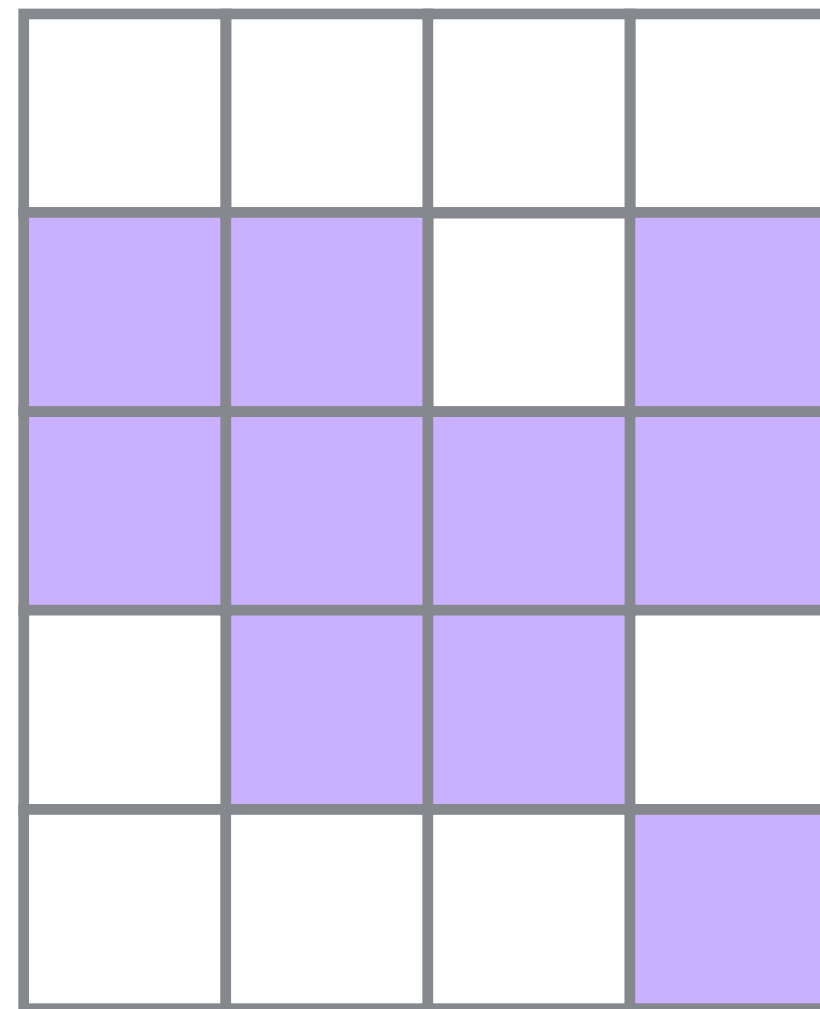
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



[심화 예제] 뒤집기

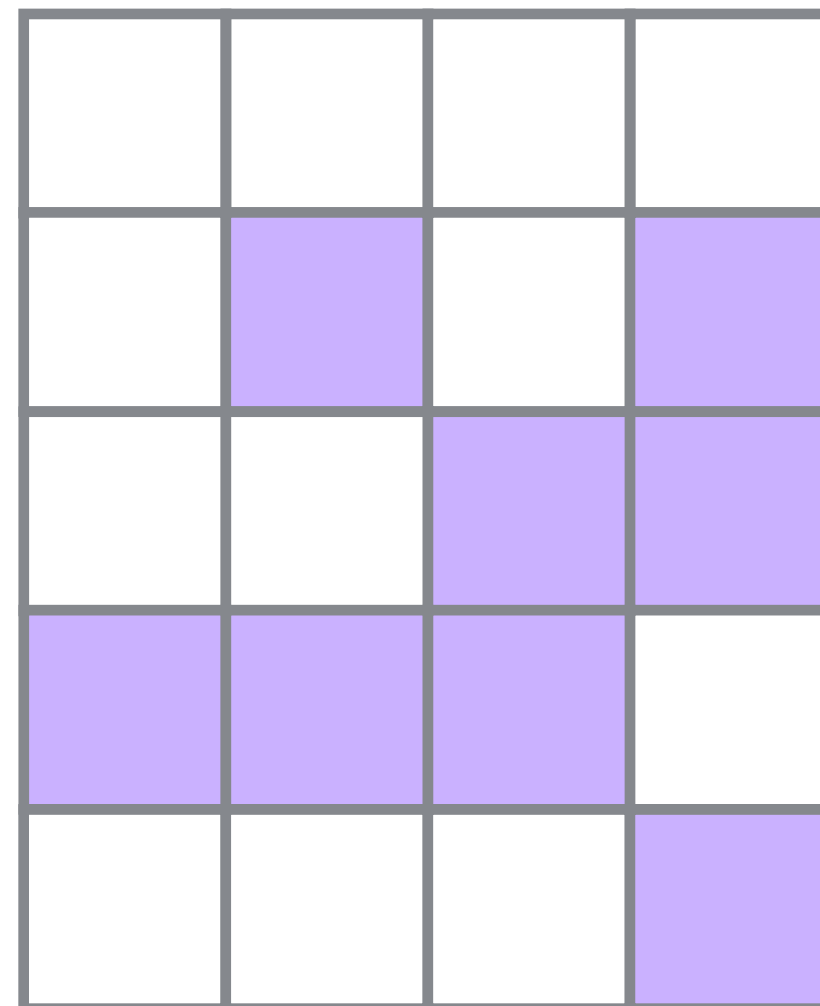
관찰 2 : + 모양임을 이용할 수 있다





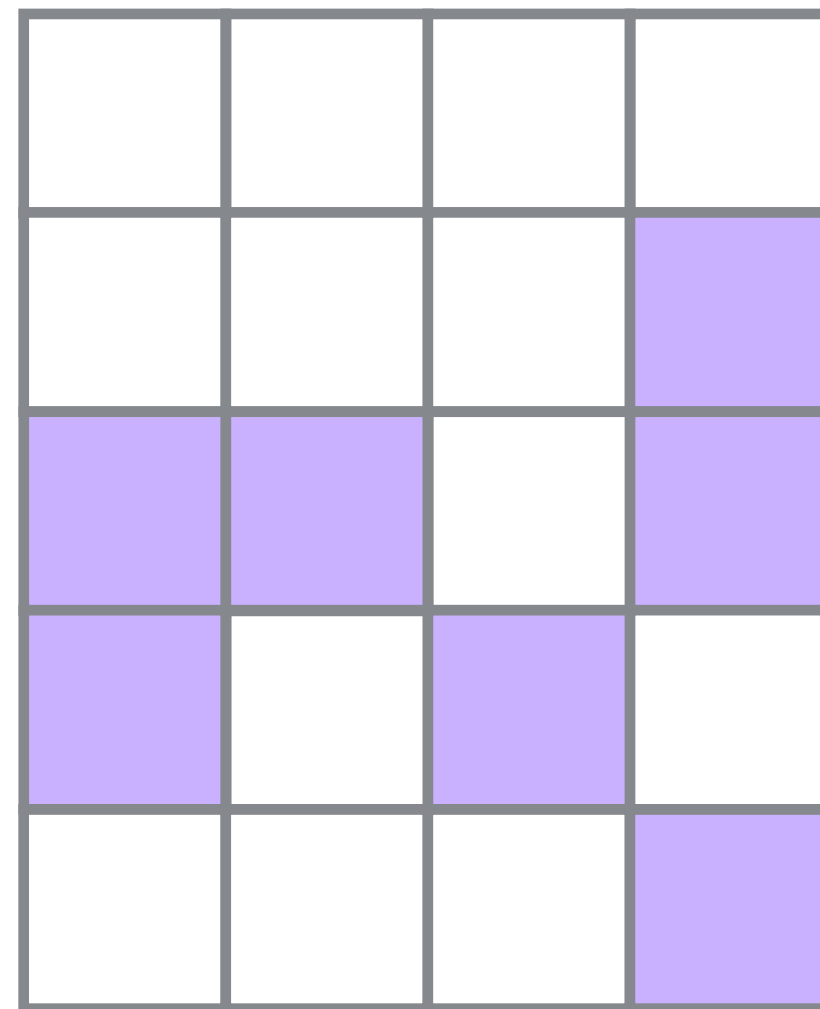
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



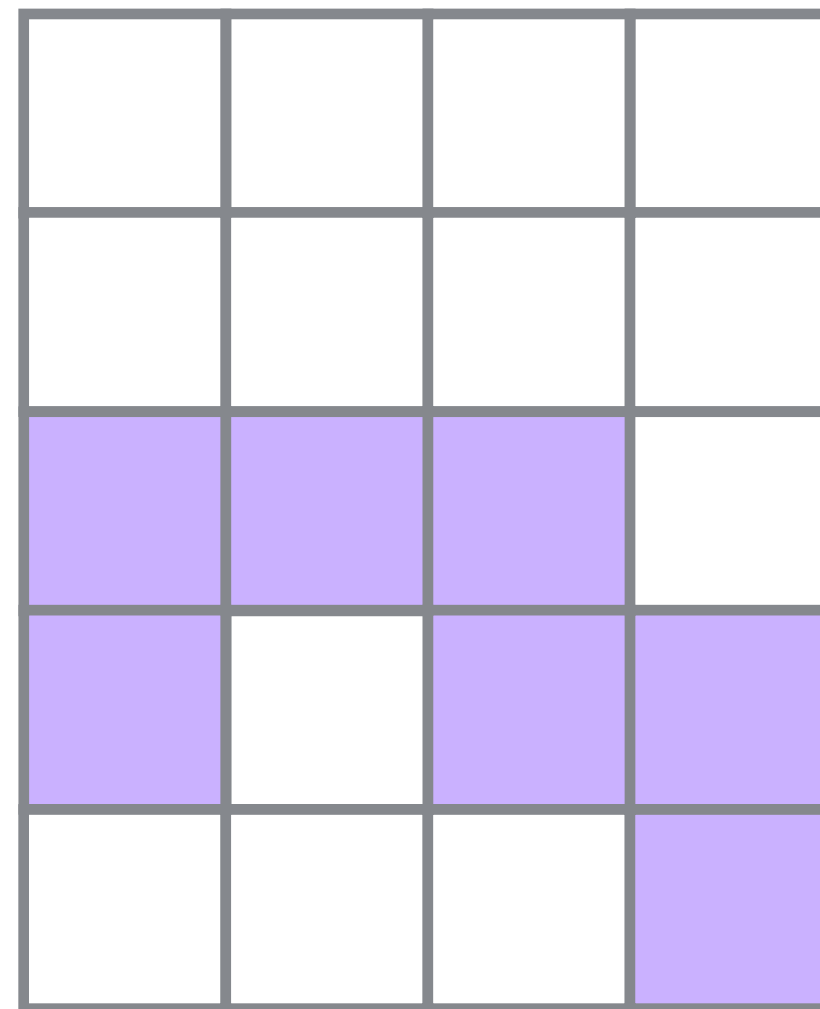
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



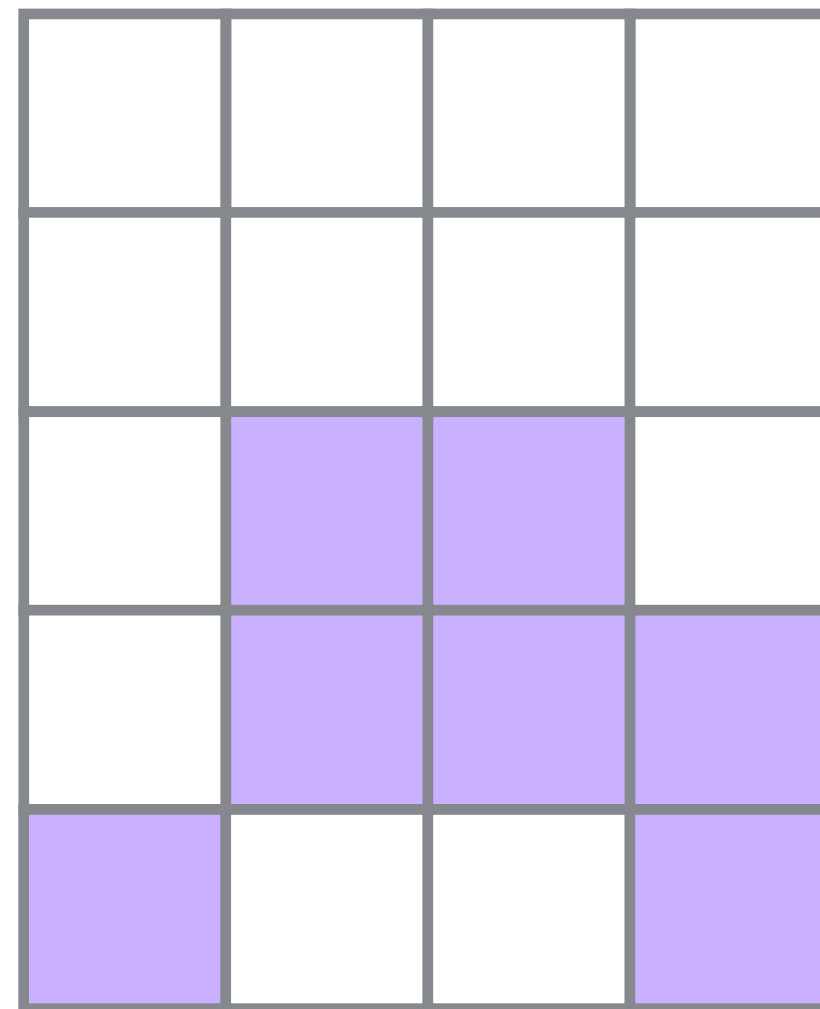
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



[심화 예제] 뒤집기

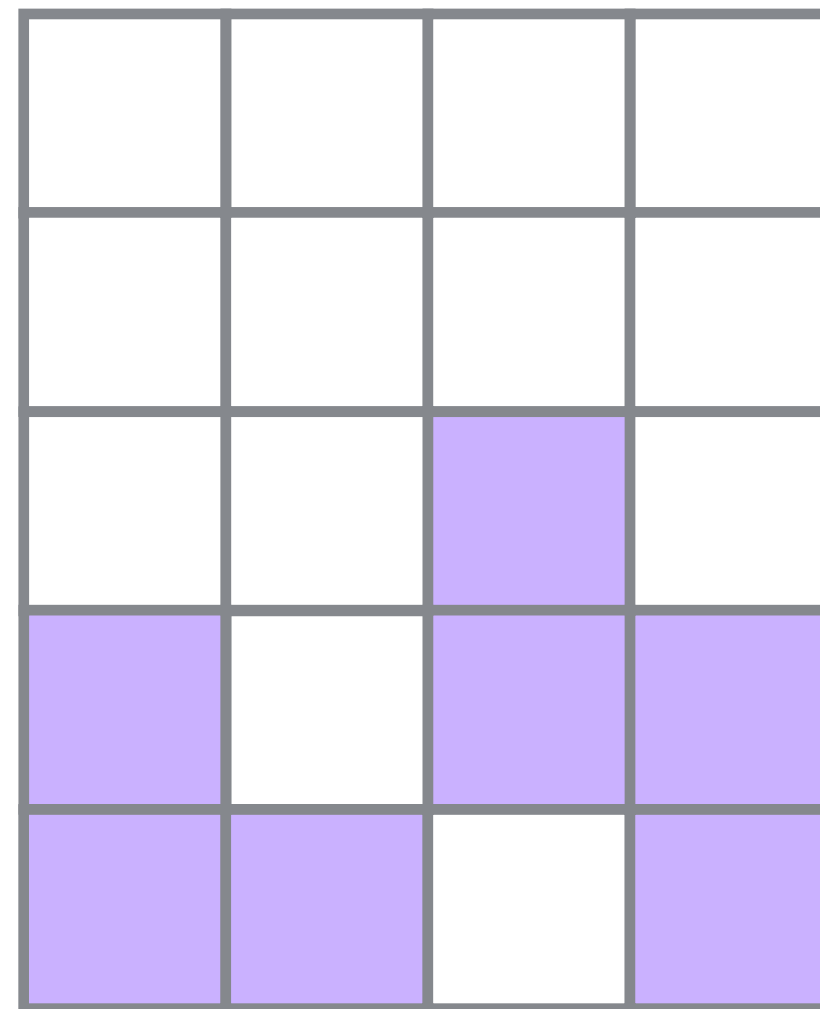
관찰 2 : + 모양임을 이용할 수 있다





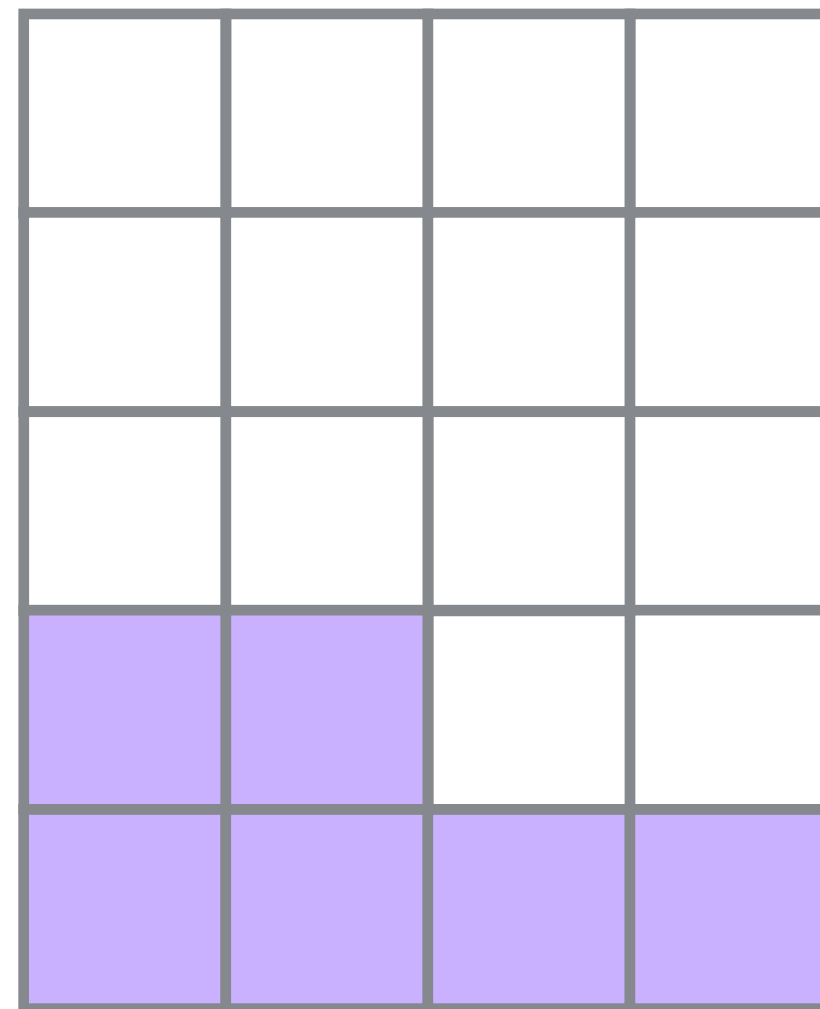
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



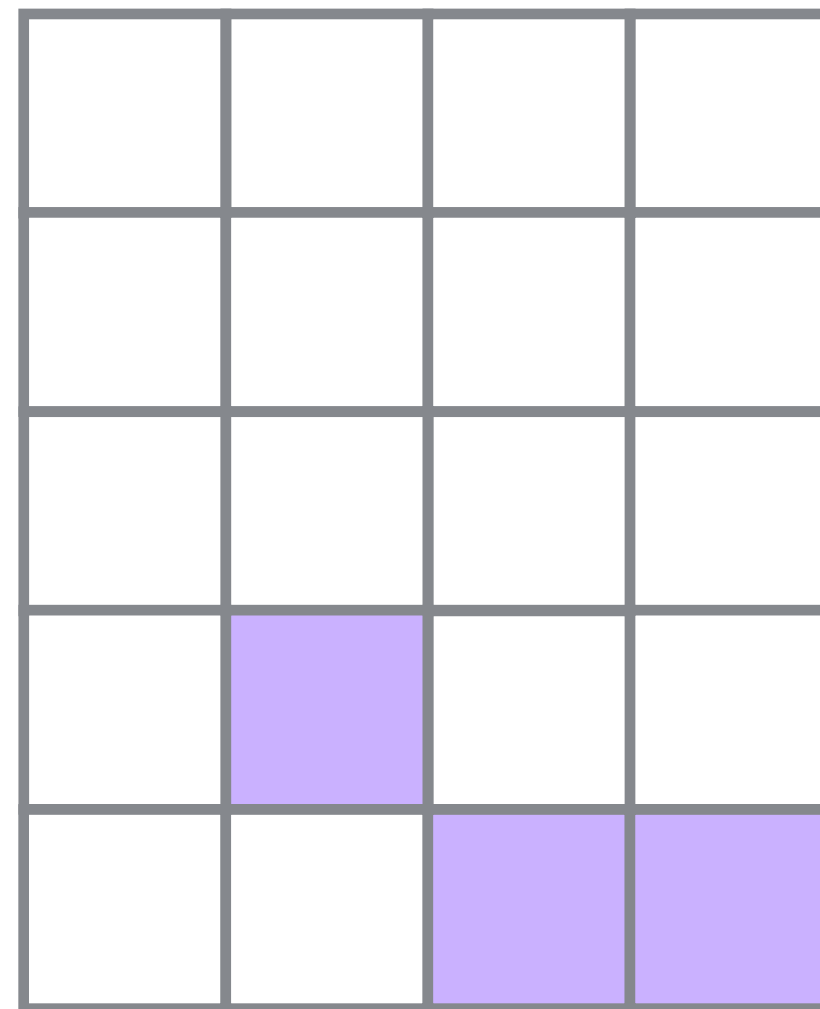
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



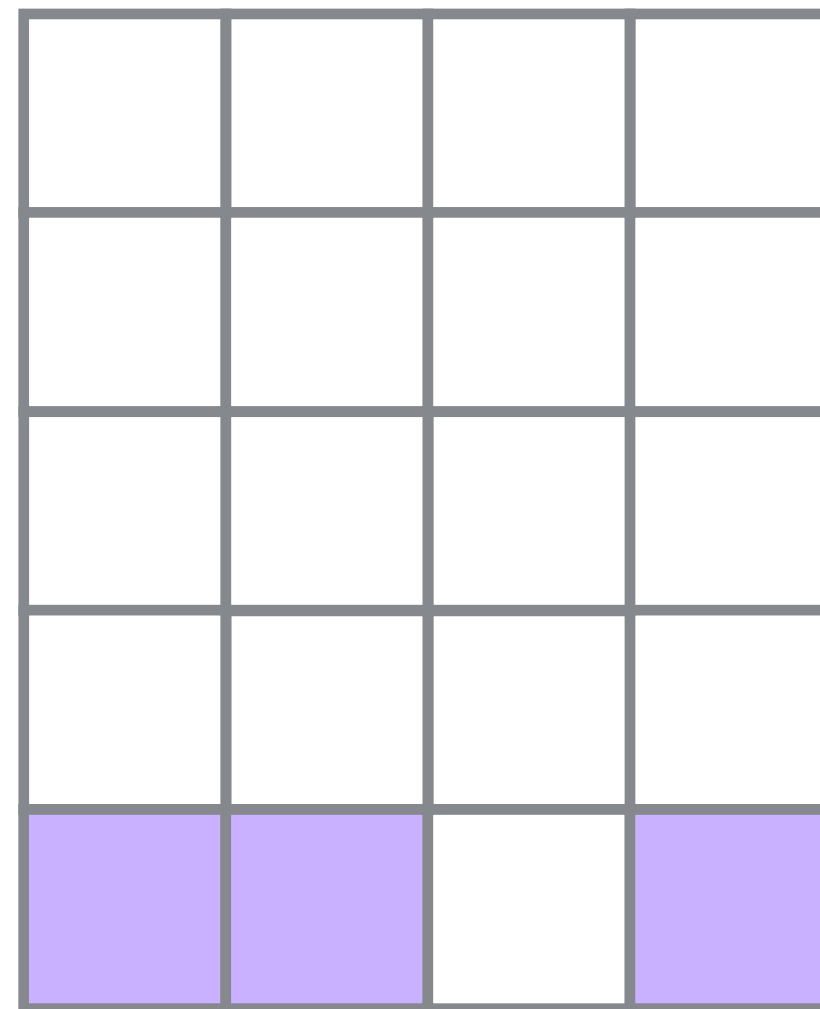
[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



[심화 예제] 뒤집기

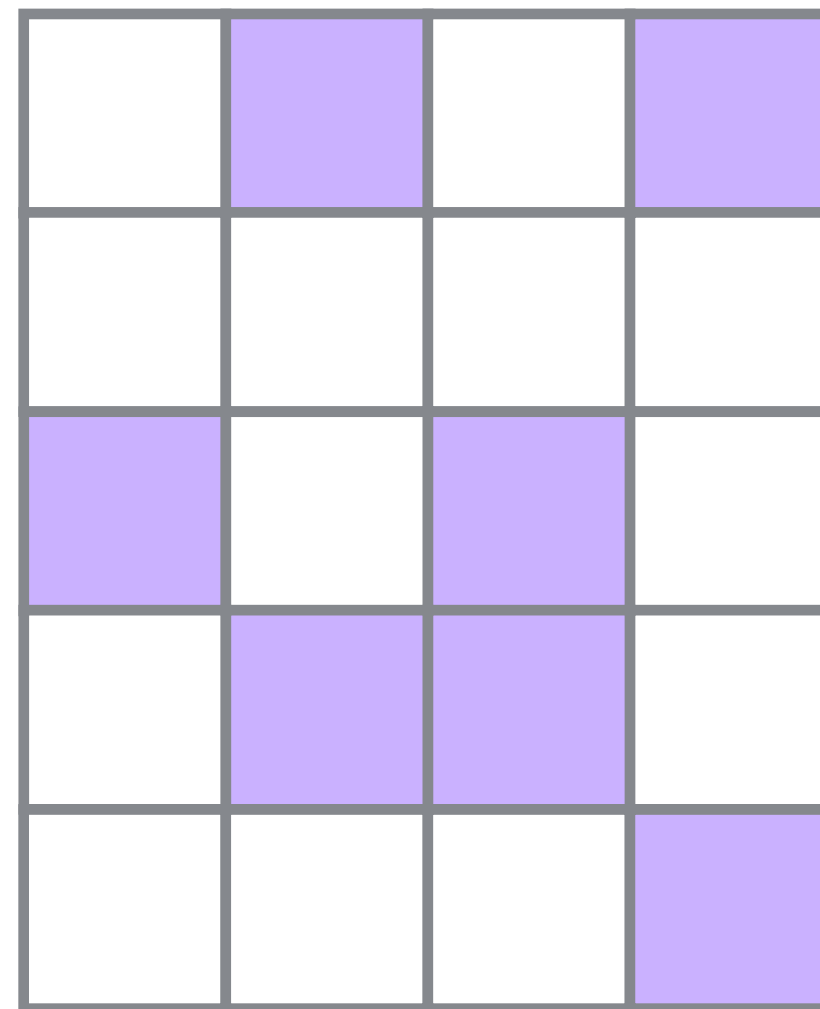
관찰 2 : + 모양임을 이용할 수 있다



IMPOSSIBLE

[심화 예제] 뒤집기

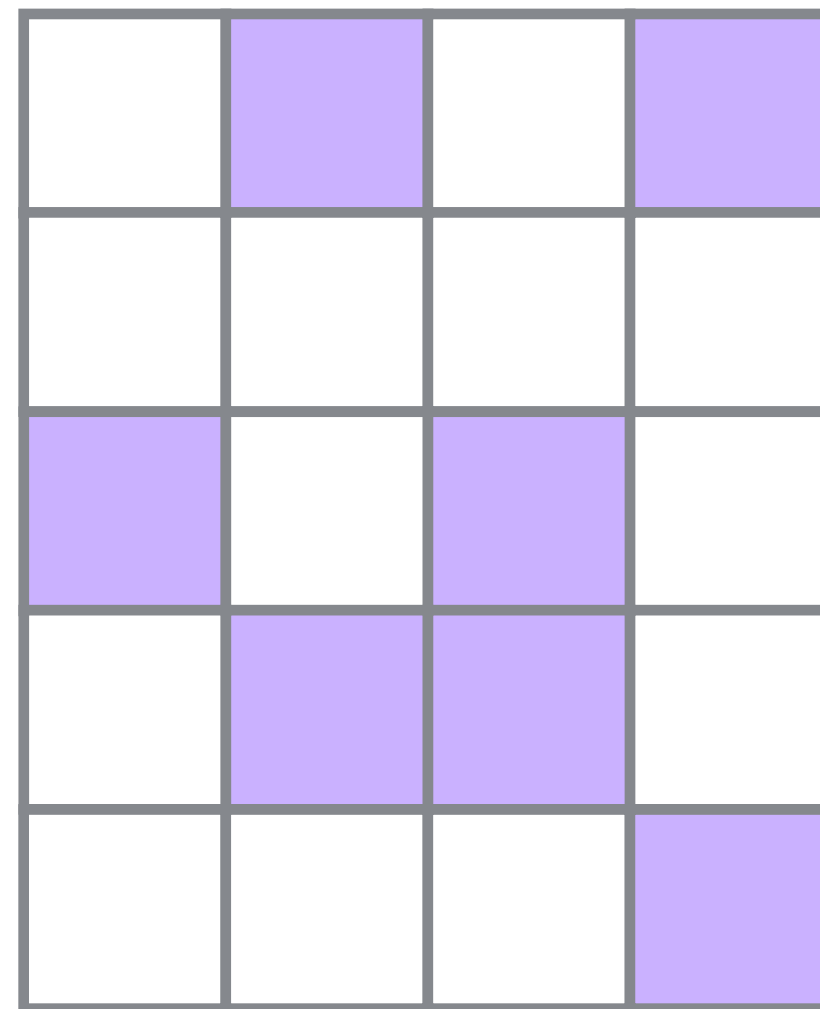
관찰 2 : + 모양임을 이용할 수 있다



첫 번째 줄만 모든 경우를 둘러보면, 두 번째 줄부터는 정해져있음

[심화 예제] 뒤집기

관찰 2 : + 모양임을 이용할 수 있다



$O(2^M) \times O(NM)$



생각해볼 점

각 문제마다 풀이의 시간복잡도가 다르다

연속부분 최대합: $O(n^3)$, 균형 맞추기: $O(2^n)$

내 풀이가 얼마나 좋은 풀이인가 ?

예를 들어, 균형 맞추기는 $O(n^3)$ 에 불가능한가 ?

Complexity Theory



Complexity Theory

문제 자체에도 복잡도가 존재한다

빨리 풀 수 있는 문제가 있고, 그렇지 않은 문제가 있다

(정확하진 않지만 그렇게 틀린 말도 아닌 초 단순한 개념)

P class

다항 시간 내에 해결 가능한 문제들의 집합

NP-Complete class

다항 시간 내에 해결이 불가능한 문제들의 집합

알고리즘 과정에서 다루는 문제들

(거의 대부분) P 문제들만을 다룹니다

가능한 모든 경우는 2^n 개 처럼 보이지만 실제로는
적은 수의 경우만 고려하면 충분한 문제만을 다룹니다

알고리즘에서는 고려해야 하는 경우를 줄이는 방법을 배웁니다

어떻게 좀 더 줄일지에 대한 고민을 계속해서 하게 됩니다

하지만 대표적인 NP-Complete 문제는 알면 좋습니다

NP-Complete 로 유명한 문제를 굳이 다항시간에 풀려고 하는
일을 미연에 방지할 수 있기 때문입니다

비트마스크



비트마스크란?

- 2진수로 표현된 1101

- $2^3 \times 1 + 2^2 \times 1 + 2^1 \times 0 + 2^0 \times 1 = 13$

- 2진수의 특성

- 2진수는 0과 1만의 값을 가지고, true/false 상태를 가짐
 - 모든 10진수는 2진수로 표현이 가능(역도 성립)
 - bool 자료형 배열보다 메모리를 덜 차지하고, 효율적

그래서 비트마스크는 왜 사용하는 걸까?



비트마스크를 사용하는 이유

길이가 5인 집합 $A = [1, 2, 3, 4, 5]$ 에 대해 부분 집합을 표현한다면?

```
{1, 2, 3, 4} -> [1, 1, 1, 1, 0]
{1, 2, 4}    -> [1, 1, 0, 1, 0]
{1, 3}       -> [1, 0, 1, 0, 0]
```

→ 단순 배열로 구현 시, 메모리 차지 및 오버헤드 증가

부분 집합을 배열이 아닌 정수를 통해 표현 가능

```
{1, 2, 3, 4} -> 01111 ->  $(2^3 * 1) + (2^2 * 1) + (2^1 * 1) + (2^0 * 1) = 15$ 
{1, 2, 4}    -> 01011 ->  $(2^3 * 1) + (2^1 * 1) + (2^0 * 1) = 11$ 
{1, 3}       -> 00101 ->  $(2^2 * 1) + (2^0 * 1) = 5$ 
```

→ 공간 효율성 향상

부분 집합에서 i 를 추가하고 싶으면, 단순히 i 번째 비트의 값을 1로 변경하는 방식 등으로 구현



• AND(&)

- 대응하는 비트들이 모두 1일 때, 1 반환
- 예시) $1010 \& 1111 = 1010$

• OR(|)

- 대응하는 비트들이 하나 이상 1일 때, 1 반환
- 예시) $1010 | 1111 = 1111$

• XOR(^)

- 대응하는 두 비트가 서로 다르면 1 반환
- 예시) $1010 \wedge 1111 = 0101$

• NOT(~)

- 비트의 값을 반전하여 반환
- 예시) $\sim 1010 = 0101$

• 시프트(Shift)연산(<<, >>)

- 왼쪽이나 오른쪽으로 비트를 해당 횟수만큼 옮김
- 예시 1) $(00001010 \ll 2) = 00101000$
- 예시 2) $(00001010 \gg 2) = 00000010$



특정 비트 변경

- i 번째 비트를 1로 변경하려면?
 - OR 1
- i 번째 비트를 0으로 변경하려면?
 - AND 0
- 비트 연산자를 같이 사용
 - i 번째 비트를 변경하기 위해 시프트 연산자($<<$)와 같이 사용
- 실제 알고리즘 문항에서는 i 번째 정점(노드)을 방문했는지 체크 등에 활용
 - 1, 3번째 정점을 방문하면 $\rightarrow 0101(5)$